

→ Binary Tree-3

Kth level of a tree

### Kth Level of a Tree

**Approach**  
 $K = 3$

$O(N)$

Preorder

4 5 6 7

```
Klevel (root, level, k)
{
    if (root == null)
        return;

    if (level == k) {
        print (root.data) → return;
    }

    Klevel (root.left, level+1, k);
    Klevel (root.right, level+1, k);
}
```

→ Lowest Common Ancestor

Ancestor: node to root

### Lowest Common Ancestor

**Approach**  $n1 = 4, n2 = 6$

path1 (n1): 1 | 2 | 4

path2 (n2): 1 | 3 | 6

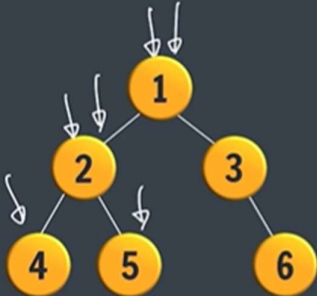
lca = 1

## Lowest Common Ancestor

Approach

$n1 = 4, n2 = 5$

path1(n1): 1 2 4  
path2(n2): 1 2 5  
lca = 2

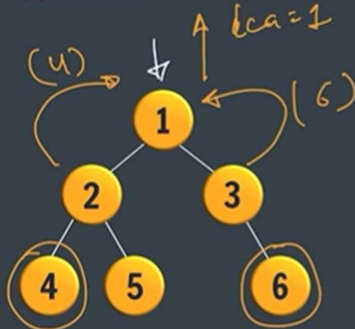


→ Approach-2

## Lowest Common Ancestor

Approach 2

$n1 = 4, n2 = 6$



① root = null  
root = n1  
root = n2  
→ return root

left LCA (4)

right LCA

②

right LCA = null → left LCA

left LCA = null → right LCA

right & left != null → return root

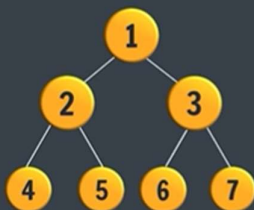
→ Min distance between nodes

## Min Distance between nodes

Approach

$n1 = 4, n2 = 6$

lca



① dist1 lca to n1

② dist2 lca to n2

dist1 + dist2 = min. dist btw n1 & n2

APNA  
COLLEGE

## Min Distance between nodes

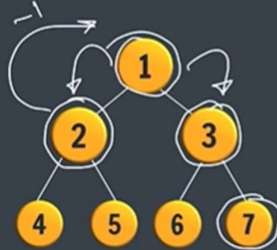
Approach

$n1 = 4$   $n2 = 6$

① dist1 lca to n1

② dist2 lca to n2

$dist1 + dist2 = \text{min. dist btw } n1 \& n2$



if (root == null)  
return -1

if (root.data == n)  
return 0;

left n search  
right n search

-1, -1 → -1  
valid (>-1) → value + 1

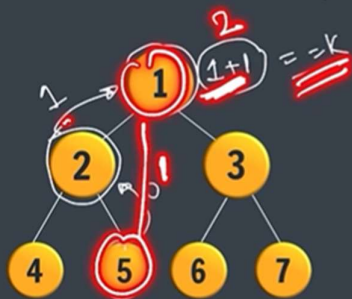
jcreddy2096@gmail.com

→ Kth ancestor of node

## Kth Ancestor of node

Approach

$n = 5$   
 $k = 2$



① find my node

② if (root.data == node)  
return 0;

left Dist ✓

right Dist ✓

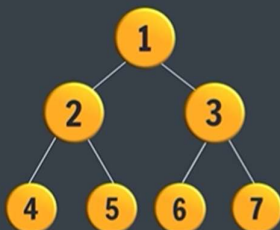
-1, -1 → -1

valid value →  $(dist + 1) == k$   
print root.data

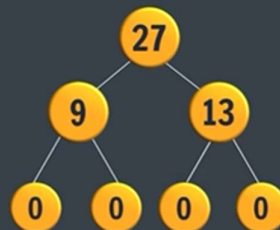
→ Transform to Sum Tree

## Transform to Sum Tree

sum



→

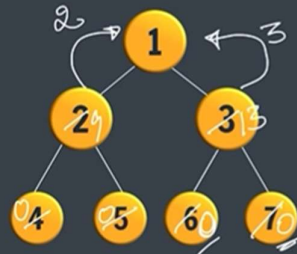


LS data + RS data

Each node = sum of left & right subtrees

## Transform to Sum Tree

### Approach



LC = 2  
RC = 3  
data =

```
if (root == null) return 0;
```

```
leftChild = transform(root.left);
```

```
rightChild = transform(root.right);
```

```
data = root.data;
```

```
root.data = root.left + leftChild +  
            root.right + rightChild;
```

```
return data;
```